

UNIVERSITATEA TEHNICĂ A MOLDOVEI

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

LUCRARE DE LABORATOR NR. 4

The xv6 Shell from the Outside

Disciplina: Sisteme de Operare

A efectuat:
student gr. FAF-243
Daniel Nicorici

Au verificat:
asist. univ. Patricia Reițman
asist. univ. Marius Dumitrașcu

Chișinău — 2026

Cuprins

Purpose of the work **2**

1 Part 1. Fix sleep for the current xv6 **2**

2 Part 2. bash versus the xv6 shell **3**

3 Part 3. Who expands *? A program that prints its argv **5**

4 Part 4. Read the shell **7**

Conclusions **9**

Purpose of the work

The whole user interface of xv6 is one program, `user/sh.c`. The purpose of this lab is to use it side by side with bash and find out, by experiment, what a shell must do and what bash adds on top; to write a small program that prints the arguments it receives, which shows exactly who expands `*` and who removes quotes; and to read the parts of `sh.c` that explain what was observed. First, the `sleep` program from Lab 2 is checked against the current xv6.

The work was done on my own laptop running Arch Linux: xv6 (`mit-pdos/xv6-riscv`, commit `06aad25`) cross-compiled with `riscv64-linux-gnu-gcc` 16.2.1 and run on QEMU 11.1.1, and on the Linux side GNU bash 5.3.15 and gcc 16.1.1.

1 Part 1. Fix sleep for the current xv6

In August 2025 the xv6 authors renamed the system call `sleep()` to `pause()`. My tree declares only `pause`, and my `sleep.c` from Lab 2 already calls `pause()`, so it builds without changes; `$U/_sleep\` is on line 153 of the Makefile.

```
$ cd ~/xv6-riscv
$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

Figure 1.1 – The grep result in user.h

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));
  exit(0);
}
```

Figure 1.2 – The updated sleep.c

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -
    ↪ global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device
    ↪ virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ sleep 20
$ sleep
usage: sleep <ticks>
```

Figure 1.3 – sleep running in xv6, with and without an argument

`sleep 20` pauses for 20 ticks (about 2 seconds) before the prompt returns; without an argument the program prints its usage message to standard error and exits with status 1.

2 Part 2. bash versus the xv6 shell

Every command was run twice: in bash, in an empty folder `~/shtest` containing only `README`, and at the xv6 prompt. Table 2.1 summarises what each shell printed.

Table 2.1 – The same commands in bash and in the xv6 shell

Command	Tests	bash	xv6
`echo hello > f ; cat < f`	redirection	`hello`	`hello`
`ls | grep READ`	pipe	`README`	`README 2 2 2441`
`(echo a; echo b) | wc`	grouping, list	`2 2 4`	`2 2 4`
`echo one ; echo two`	sequence	`one`, `two`	`one`, `two`
`echo bg &`	background	`[1] 135185`, `bg`, later `[1]+ Done`	`bg`, no job number
`ls *`	globbing	`README f`	`ls: cannot open *`
`echo $HOME`	variables	`/home/Nico`	`$HOME`
`echo "a b"`	quotes	`a b`	`"a b"`
`mkdir d ; cd d ; ls ..`	PATH	`README d f`	`exec cd failed`, then `ls` lists `/`; after a separate `cd d`: `exec ls failed`
arrow up, Tab, Ctrl+R	history, completion	recalls, completes, searches	`exec ^[[A failed`, `exec ech failed`, `exec ^Rtwo failed`

```
$ mkdir ~/shtest && cd ~/shtest && touch README
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README
$ (echo a; echo b) | wc
      2       2       4
$ echo one ; echo two
one
two
$ echo bg &
[1] 135185
$ bg
ls *
```

```
README  f
[1]+  Done                        echo bg
$ echo $HOME
/home/Nico
$ echo "a b"
a b
$ mkdir d ; cd d ; ls ..
README  d  f
$ mkdir d ; cd d ; ls ..
d
$ echo  one
one
(reverse-i-search)`two': echo one ; echo two
one
two
```

Figure 2.1 – The commands run in bash

In bash the background `echo` printed `bg` only after the next prompt had appeared, so it shows up after `$`, and the job's `Done` line came after the following command. The last three commands test the keyboard: the arrow-up key recalled the previous line, and running it again created `d/d` and listed it; typing `ech` and Tab completed the word to `echo` plus a space; Ctrl+R followed by `two` found `echo one ; echo two` in the history and ran it again.

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
$ ^[[A
exec ^[[A failed
$ ech[Tab] one
exec ech failed
$ ^Rtwo
exec ^Rtwo failed
```

Figure 2.2 – The same commands run in xv6

In xv6 the special keys are not interpreted: the arrow key arrives as the raw characters `ESC [ A` (shown as `^[[A`), Tab is just a space-like character that ends the word `ech`, and Ctrl+R is the raw character `^R`. The shell treats each of them as part of a program name and fails to run it.

```
$ mkdir d ; cd d ; ls ..
exec cd failed
```

```
.              1 1 1024
..             1 1 1024
README         2 2 2441
[... 23 entries omitted ...]
f              2 26 6
d              1 27 32
$ cd d
$ ls ..
exec ls failed
$ /ls ..
.              1 1 1024
..             1 1 1024
README         2 2 2441
[... 23 entries omitted ...]
f              2 26 6
d              1 27 32
$ cd /
```

Figure 2.3 – ls failing after cd d, and the working alternative

Observations:

1. **Common features and bash-only features.** Both shells run programs with arguments, redirect input and output with `<` and `>`, connect programs with `|`, run sequences with `;`, group commands with `( )` and start background commands with `&`. Only bash has globbing, variables, quoting, a PATH search, `cd` anywhere on the line, job numbers and `Done` messages, command history, Tab completion and Ctrl+R search.
2. **Why `ls` stops working after `cd d`.** The xv6 shell passes the program name straight to `exec`, which looks it up relative to the current directory, and there is no PATH. All xv6 programs live in the root directory `/`, so after `cd d` the name `ls` no longer points to a file and the shell prints `exec ls failed`. The fix is to give the full path, `/ls ..` (or `../ls ..`). The first line also shows a second limitation: `cd` inside a `;` list is not recognised and fails with `exec cd failed` (explained in Part 4).
3. **What I would miss most, and what is easiest to add.** In daily work I would miss history and Tab completion the most, because they save the most typing. The easiest feature to add is a PATH-like fallback: when `exec` fails and the name contains no `/`, the shell could retry with `/` in front of the name, which takes only a few lines in `runcmd`.

3 Part 3. Who expands *? A program that prints its argv

```
$ cat user/args.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
$ grep -n "_sleep\|_args" Makefile
```

```
153:    $U/_sleep\
154:    $U/_args\
```

Figure 3.1 – args.c and the UPROGS line

```
$ args one              two         three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
```

Figure 3.2 – args output in xv6

```
$ cat args_linux.c
#include <stdio.h>

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
$ gcc -Wall -o args args_linux.c
$ ls *.c
args.c  args_linux.c  sleep.c
$ ./args one              two         three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [args.c]
argv[2] = [args_linux.c]
argv[3] = [sleep.c]
argv[4] = [x y]
```

```
$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/Nico]
$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 3.3 – args_linux output in bash

Observations:

1. **`args *.c "x y"` in both shells.** In xv6 the program received 4 arguments: `args`, `*.c`, `"x` and `y"`. In bash it received 5: `./args`, the three file names `args.c`, `args_linux.c`, `sleep.c`, and `x y` as a single argument without quotes.
2. **Who expands `*` and removes quotes.** The shell. The program is the same in both cases and only prints what it is given, and the kernel's `exec` only copies the argument strings it receives. Bash replaced `*.c` with the matching names and removed the quotes before calling `exec`; the xv6 shell does neither, so the program received the raw text. The same happened with `$HOME` (expanded only by bash) and `'single quoted'` (one argument in bash, two in xv6).
3. **The extra spaces in `one two three`.** They disappeared in both shells: both split the line at runs of whitespace, so spaces only separate arguments and are never part of them, and both programs received `argc = 4`.

4 Part 4. Read the shell

```
$ wc -l user/sh.c
499 user/sh.c
$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
$ grep -n "symbols\[\]\|whitespace\[\]" user/sh.c
268:char whitespace[] = " \t\r\n\v";
269:char symbols[] = "<|>&;()";
```

Figure 4.1 – The command kinds defined in sh.c

```
$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
```

```
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
```

Figure 4.2 – Where each kind is run

Lines 75–125 are `runcmd()`, which executes each kind; lines 284–294 are the tokenizer `gettoken()`, which recognises the special characters; lines 391–397 parse redirections; and lines 469–493 are `nulterminate()`. Searching for `cd` finds only the error message on line 170, because the check compares single characters:

```
$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

```
159   // Read and run input commands.
160   while (getcmd(buf, sizeof(buf)) >= 0) {
161     char *cmd = buf;
162     while (*cmd == ' ' || *cmd == '\t')
163       cmd++;
164     if (*cmd == '\n') // is a blank command
165       continue;
166     if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
167       // Chdir must be called by the parent, not the child.
168       cmd[strlen(cmd) - 1] = 0; // chop \n
169       if (chdir(cmd + 3) < 0)
170         fprintf(2, "cannot cd %s\n", cmd + 3);
171     } else {
172       if (fork1() == 0)
173         runcmd(parsecmd(cmd));
174       wait(0);
175     }
176   }
```

Figure 4.3 – The code that handles cd

```
$ grep -n "'\*'\|'\$'\|glob\|getenv" user/sh.c || echo "(no matches)"
(no matches)
```

Figure 4.4 – Searching sh.c for code that handles * or $

Observations:

1. **The five kinds of command.** `EXEC` is a plain command, produced by ordinary words with no special character (`ls ..`); `REDIR` is produced by <, > and >> (which the tokenizer turns into +); `PIPE` by `|`; `LIST` by `;`; and `BACK` by `&`. Parentheses only group commands and are not a kind of their own.

2. **Where `cd` is handled, and why not as a program.** In `main()`, lines 166–171: if a line starts with `cd `, the shell itself calls `chdir` and does not fork. The current directory belongs to a process, so if a child process changed directory, the change would disappear when the child exits and the shell would stay where it was. Because the check only looks at the start of the line, `mkdir d ; cd d ; ls ..` goes to `runcmd()` instead, where `cd` is treated as a program and fails with `exec cd failed`, exactly as seen in Part 2.

3. **Code for `*` or `$`.** There is none. The only special characters the shell knows are `symbols[] = "<|>&;()"` plus whitespace. This explains Part 2: `ls *` received a literal `*`, `echo $HOME` printed `$HOME`, and quotes stayed inside the arguments because they are ordinary characters to this shell.

Conclusions

Running the same commands in two shells separated what a shell must do from what bash adds for comfort. The essential part is small: read a line, split it into words, recognise a handful of special characters, and turn them into the five kinds of command that `sh.c` defines, which it then runs with `fork`, `exec`, `pipe`, `dup` and `chdir`. In 499 lines of C, xv6 already supports programs, redirection, pipes, sequences, grouping and background jobs.

Everything else in bash, such as globbing, variables, quotes, PATH search, history and completion, is convenience implemented inside the shell, not in the programs and not in the kernel. The `args` program proved this directly: the same code printed `*.c` and the quotes literally under xv6, and the expanded file names under bash. The experiment with `cd` showed the one thing a shell cannot delegate to a child process, because the current directory belongs to the process that changes it. Finally, the renaming of `sleep` to `pause` was a reminder that the interface between user programs and the kernel is defined by `user.h`, and code has to follow it when it changes.